

Aula 1 (Módulo 17) — PowerShell Ofensivo Avançado

Lembrete do `CLAUDE.md` deste repositório: técnicas ofensivas aqui existem só para você reconhecer evidência e construir detecção. Qualquer prática hands-on é só em laboratório próprio/autorizado.

1. Revisão — o que já sabemos (Módulo 02, Aula 2)

Já vimos `-EncodedCommand`, `-WindowStyle Hidden`, `-ExecutionPolicy Bypass`. Um atacante mais avançado sabe que essas flags **também** são conhecidas pelos defensores (é exatamente por isso que você aprendeu a reconhecê-las!) — então técnicas mais sofisticadas tentam contornar justamente esse tipo de detecção.

2. AMSI — a barreira que o Windows moderno já tem

AMSI (*Antimalware Scan Interface*) é uma funcionalidade do Windows que permite que o antivírus (Módulo 02, Aula 3) **inspecione o conteúdo de scripts PowerShell** mesmo depois de decodificados/desofuscados em memória — resolvendo justamente a limitação que o Script Block Logging (Module 02, Aula 5) também ataca, mas do lado de **bloqueio** em vez de só registro.

Por que atacantes tentam contornar (AMSI Bypass): existem técnicas publicamente documentadas que tentam desativar ou "cegar" o AMSI antes de executar o payload real — normalmente manipulando, via reflection do .NET, uma variável interna que controla se o AMSI está ativo para aquela sessão. Microsoft e fabricantes de EDR atualizam continuamente as defesas contra as variantes conhecidas dessa técnica — é um jogo de gato e rato ativo.

Detecção: o próprio evento de "AMSI foi desativado/bypassado" pode, em ambientes bem instrumentados, gerar seu próprio alerta — e a tentativa de bypass em si costuma deixar rastro na command line (padrões reconhecíveis de manipulação de reflection do .NET, mesmo ofuscados).

3. Além do Base64 — outras formas de ofuscação

Já vimos `-enc` (Base64). Outras técnicas comuns:

- **Concatenação de string:** quebrar um comando/palavra suspeita em pedaços concatenados em tempo de execução (ex.: juntar `"I" + "EX"` para formar `IEX`, o alias de `Invoke-Expression`), dificultando detecção por assinatura simples de texto.
- **Substituição de caracteres:** usar variáveis de ambiente, concatenação com `-join`, ou funções de substituição de string para reconstruir comandos, sempre com o mesmo objetivo: parecer diferente de uma assinatura conhecida enquanto o **comportamento** final é o mesmo.
- **Compressão:** comandos comprimidos (ex.: com `GZipStream`) antes de serem codificados em Base64 — adiciona uma camada extra antes de chegar ao texto legível.

Por que a detecção comportamental (IOA, Módulo 12) vence aqui: não importa quantas camadas de ofuscação existam, o **resultado final** ainda precisa fazer algo observável — chamar `IEX`, baixar algo da rede, criar um processo filho. É por isso que detecção baseada em comportamento (Event ID 4104 mostrando o script **já decodificado**, Módulo 02 Aula 5) é muito mais resiliente do que tentar cobrir cada variação de ofuscação individualmente.

4. Download Cradles — baixando e executando sem tocar o disco

Um **download cradle** é um padrão de comando que baixa conteúdo da internet e o executa **diretamente na memória**, sem nunca gravar um arquivo no disco — reduzindo o rastro de arquivo (Módulo 16) que um antivírus tradicional baseado em arquivo detectaria.

O padrão geral (sem entrar em sintaxe exploitable específica) usa classes do .NET como `Net.WebClient` ou `Net.Http.HttpClient` para buscar conteúdo remoto, seguido de `Invoke-Expression` (ou equivalente) para executá-lo imediatamente como código PowerShell.

Evidência que ainda existe, mesmo sem arquivo em disco: - Sysmon Event ID 3 (conexão de rede) — o download em si. - Sysmon Event ID 22 (DNS) — a resolução do domínio de origem. - Event ID 4104 (Script Block Logging) — o conteúdo do script, mesmo que ele nunca tenha existido como arquivo.

Isso reforça um ponto central do curso todo: "sem arquivo em disco" **não** significa "sem evidência" — só significa que você precisa olhar para as fontes certas (rede, DNS, script logging), não só para o sistema de arquivos.

5. Constrained Language Mode — uma defesa que vale conhecer

O PowerShell tem um modo de operação restrito chamado **Constrained Language Mode**, que limita drasticamente o que scripts podem fazer (bloqueia chamadas diretas a APIs do Windows via .NET, por exemplo) — uma configuração de hardening que reduz muito a superfície de ataque de PowerShell ofensivo, geralmente aplicada via **AppLocker** ou **Windows Defender Application Control (WDAC)**.

```
$ExecutionContext.SessionState.LanguageMode # confirma o modo atual
```

Por que importa mencionar aqui: conhecer essa defesa ajuda a entender por que, em ambientes bem protegidos, muitas técnicas ofensivas "clássicas" de PowerShell simplesmente não funcionam — e por que atacantes avançados frequentemente tentam identificar e contornar essa restrição antes de prosseguir.

6. Conectando tudo

Ofuscação (Base64, concatenação, compressão) → dificulta detecção por
ASSINATURA de texto
AMSI Bypass → tenta neutralizar a inspeção de conteúdo em tempo real
Download Cradle → executa em memória, sem arquivo em disco

DEFESA que vence tudo isso: detecção COMPORTAMENTAL (Script Block
Logging já decodificado, Sysmon rede/DNS, EDR — Módulos 02/10/11),
reforçada por hardening (Constrained Language Mode, AppLocker/WDAC)

7. Recapitulando

- AMSI permite inspeção de conteúdo em tempo real, mesmo pós-decodificação; bypasses existem e são um jogo contínuo de atualização entre atacante e defensor.
- Ofuscação (além de Base64) tenta escapar de detecção por assinatura, mas não muda o comportamento final observável.
- Download cradles executam em memória sem arquivo — a evidência migra para rede/DNS/script logging, não desaparece.
- Constrained Language Mode + AppLocker/WDAC são defesas de hardening que reduzem a superfície de ataque de PowerShell antes mesmo de qualquer detecção entrar em ação.

Próxima aula

Credential Dumping — como credenciais são extraídas da memória de um sistema comprometido, e como detectar esse acesso específico.

Fontes recomendadas

- **Microsoft Learn — "Antimalware Scan Interface (AMSI)"**: documentação oficial. <https://learn.microsoft.com/windows/win32/amsi/antimalware-scan-interface-portal>
- **Microsoft Learn — "PowerShell Constrained Language Mode"**: documentação oficial. https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about_language_modes
- **MITRE ATT&CK — T1027 (Obfuscated Files or Information) e T1562.001 (Impair Defenses)**: técnicas oficiais associadas a ofuscação e bypass de defesas. <https://attack.mitre.org/techniques/T1027/>